SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Dr. Hemavathi

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	**5**
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	**5**
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	**5**
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	**5**
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	**5**
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	**5**
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	**5**
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	**5**

9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	**5**
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	**5**

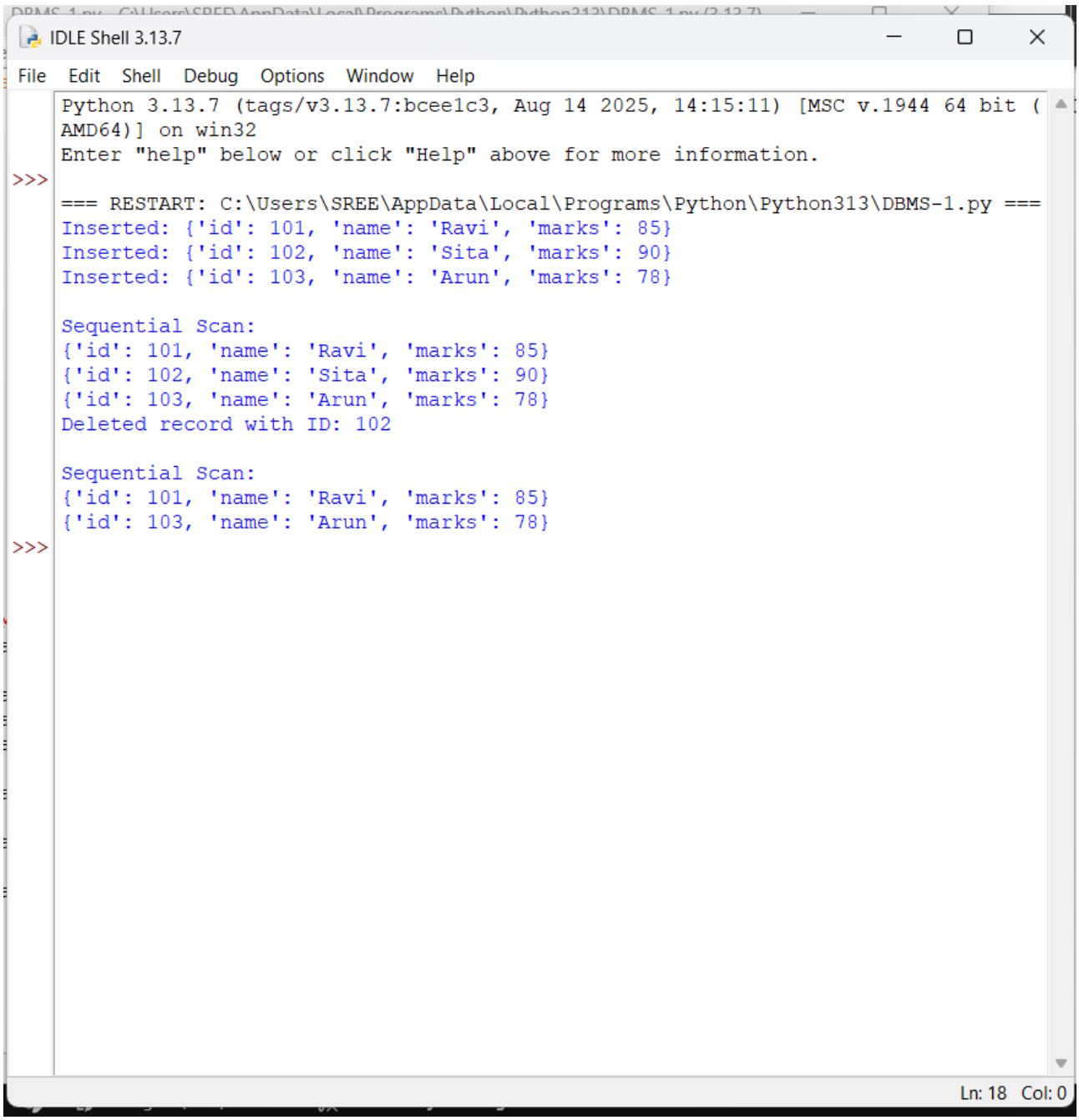
Code of Conduct:

I Sree B (192524023) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Sree B

Question:1 Write a Python program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

```
class HeapFile:
    def __init__(self):
        self.records = []

    def insert(self, record):
        self.records.append(record)
        print("Inserted:", record)

    def delete(self, record_id):
        for record in self.records:
            if record["id"] == record_id:
                self.records.remove(record)
                print("Deleted record with ID:", record_id)
                return
        print("Record not found.")

    def sequential_scan(self):
        print("\nSequential Scan:")
        for record in self.records:
            print(record)


# Main program
heap = HeapFile()

heap.insert({"id": 101, "name": "Ravi", "marks": 85})
heap.insert({"id": 102, "name": "Sita", "marks": 90})
heap.insert({"id": 103, "name": "Arun", "marks": 78})

heap.sequential_scan()

heap.delete(102)

heap.sequential_scan()
```


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:\Users\SREE\AppData\Local\Programs\Python\Python313\DBMS-1.py ===
Inserted: {'id': 101, 'name': 'Ravi', 'marks': 85}
Inserted: {'id': 102, 'name': 'Sita', 'marks': 90}
Inserted: {'id': 103, 'name': 'Arun', 'marks': 78}
Sequential Scan:
{'id': 101, 'name': 'Ravi', 'marks': 85}
{'id': 102, 'name': 'Sita', 'marks': 90}
{'id': 103, 'name': 'Arun', 'marks': 78}
Deleted record with ID: 102
Sequential Scan:
{'id': 101, 'name': 'Ravi', 'marks': 85}
{'id': 103, 'name': 'Arun', 'marks': 78}
>>>
Ln: 18 Col: 0

Question: 2-Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

```
class StaticHash:
    def __init__(self, size=50):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, student_id):
        return student_id % self.size

    def insert(self, student_id, name):
        index = self.hash_function(student_id)
        self.table[index].append((student_id, name))

    def search(self, student_id):
        index = self.hash_function(student_id)

        for record in self.table[index]:
            if record[0] == student_id:
                return record

        return None

    def display(self):
        for i, bucket in enumerate(self.table):
            if bucket:
                print(i, ":", bucket)


# Main program
hash_table = StaticHash(50)

# Insert 500 student records
for i in range(1, 501):
    hash_table.insert(i, "Student" + str(i))

print("Student 125:", hash_table.search(125))
print("Student 450:", hash_table.search(450))

print("\nHash Table:")
hash_table.display()
```

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:\Users\SREE\AppData\Local\Programs\Python\Python313\DBMS-2.py ===
Student 125: (125, 'Student125')
Student 450: (450, 'Student450')

Hash Table:
0 : [(50, 'Student50'), (100, 'Student100'), (150, 'Student150'), (200, 'Student200'), (250, 'Student250'), (300, 'Student300'), (350, 'Student350'), (400, 'Student400'), (450, 'Studen
t450'), (500, 'Student500')]
1 : [(1, 'Student1'), (51, 'Student51'), (101, 'Student101'), (151, 'Student151'), (201, 'Student201'), (251, 'Student251'), (301, 'Student301'), (351, 'Student351'), (401, 'Student401
'), (451, 'Student451')]
2 : [(2, 'Student2'), (52, 'Student52'), (102, 'Student102'), (152, 'Student152'), (202, 'Student202'), (252, 'Student252'), (302, 'Student302'), (352, 'Student352'), (402, 'Student402
'), (452, 'Student452')]
3 : [(3, 'Student3'), (53, 'Student53'), (103, 'Student103'), (153, 'Student153'), (203, 'Student203'), (253, 'Student253'), (303, 'Student303'), (353, 'Student353'), (403, 'Student403
'), (453, 'Student453')]
4 : [(4, 'Student4'), (54, 'Student54'), (104, 'Student104'), (154, 'Student154'), (204, 'Student204'), (254, 'Student254'), (304, 'Student304'), (354, 'Student354'), (404, 'Student404
'), (454, 'Student454')]
5 : [(5, 'Student5'), (55, 'Student55'), (105, 'Student105'), (155, 'Student155'), (205, 'Student205'), (255, 'Student255'), (305, 'Student305'), (355, 'Student355'), (405, 'Student405
'), (455, 'Student455')]
6 : [(6, 'Student6'), (56, 'Student56'), (106, 'Student106'), (156, 'Student156'), (206, 'Student206'), (256, 'Student256'), (306, 'Student306'), (356, 'Student356'), (406, 'Student406
'), (456, 'Student456')]
7 : [(7, 'Student7'), (57, 'Student57'), (107, 'Student107'), (157, 'Student157'), (207, 'Student207'), (257, 'Student257'), (307, 'Student307'), (357, 'Student357'), (407, 'Student407
'), (457, 'Student457')]
8 : [(8, 'Student8'), (58, 'Student58'), (108, 'Student108'), (158, 'Student158'), (208, 'Student208'), (258, 'Student258'), (308, 'Student308'), (358, 'Student358'), (408, 'Student408
'), (458, 'Student458')]
9 : [(9, 'Student9'), (59, 'Student59'), (109, 'Student109'), (159, 'Student159'), (209, 'Student209'), (259, 'Student259'), (309, 'Student309'), (359, 'Student359'), (409, 'Student409
'), (459, 'Student459')]
10 : [(10, 'Student10'), (60, 'Student60'), (110, 'Student110'), (160, 'Student160'), (210, 'Student210'), (260, 'Student260'), (310, 'Student310'), (360, 'Student360'), (410, 'Student
410'), (460, 'Student460')]
11 : [(11, 'Student11'), (61, 'Student61'), (111, 'Student111'), (161, 'Student161'), (211, 'Student211'), (261, 'Student261'), (311, 'Student311'), (361, 'Student361'), (411, 'Student
411'), (461, 'Student461')]
12 : [(12, 'Student12'), (62, 'Student62'), (112, 'Student112'), (162, 'Student162'), (212, 'Student212'), (262, 'Student262'), (312, 'Student312'), (362, 'Student362'), (412, 'Student
412'), (462, 'Student462')]
13 : [(13, 'Student13'), (63, 'Student63'), (113, 'Student113'), (163, 'Student163'), (213, 'Student213'), (263, 'Student263'), (313, 'Student313'), (363, 'Student363'), (413, 'Student
413'), (463, 'Student463')]
14 : [(14, 'Student14'), (64, 'Student64'), (114, 'Student114'), (164, 'Student164'), (214, 'Student214'), (264, 'Student264'), (314, 'Student314'), (364, 'Student364'), (414, 'Student
414'), (464, 'Student464')]
15 : [(15, 'Student15'), (65, 'Student65'), (115, 'Student115'), (165, 'Student165'), (215, 'Student215'), (265, 'Student265'), (315, 'Student315'), (365, 'Student365'), (415, 'Student
415'), (465, 'Student465')]
16 : [(16, 'Student16'), (66, 'Student66'), (116, 'Student116'), (166, 'Student166'), (216, 'Student216'), (266, 'Student266'), (316, 'Student316'), (366, 'Student366'), (416, 'Student
416'), (466, 'Student466')]
17 : [(17, 'Student17'), (67, 'Student67'), (117, 'Student117'), (167, 'Student167'), (217, 'Student217'), (267, 'Student267'), (317, 'Student317'), (367, 'Student367'), (417, 'Student
417'), (467, 'Student467')]
18 : [(18, 'Student18'), (68, 'Student68'), (118, 'Student118'), (168, 'Student168'), (218, 'Student218'), (268, 'Student268'), (318, 'Student318'), (368, 'Student368'), (418, 'Student
418'), (468, 'Student468')]
19 : [(19, 'Student19'), (69, 'Student69'), (119, 'Student119'), (169, 'Student169'), (219, 'Student219'), (269, 'Student269'), (319, 'Student319'), (369, 'Student369'), (419, 'Student
419'), (469, 'Student469')]
20 : [(20, 'Student20'), (70, 'Student70'), (120, 'Student120'), (170, 'Student170'), (220, 'Student220'), (270, 'Student270'), (320, 'Student320'), (370, 'Student370'), (420, 'Student
420'), (470, 'Student470')]
```

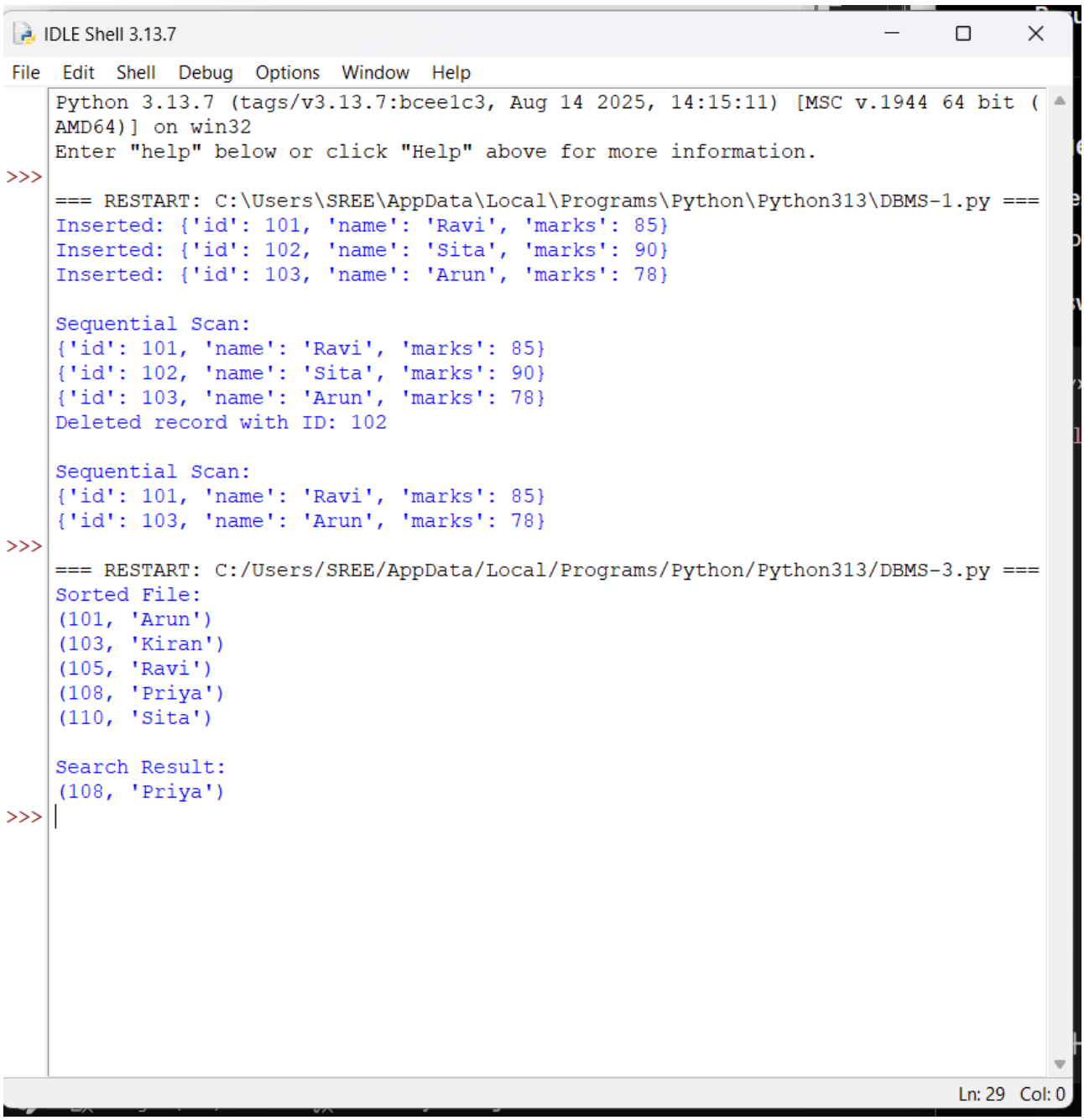
File Edit Shell Debug Options Window Help

```
425'), (475, 'Student475')]
26 : [(26, 'Student26'), (76, 'Student76'), (126, 'Student126'), (176, 'Student176'), (226, 'Student226'), (276, 'Student276'), (326, 'Student326'), (376, 'Student376'), (426, 'Student
426'), (476, 'Student476')]
27 : [(27, 'Student27'), (77, 'Student77'), (127, 'Student127'), (177, 'Student177'), (227, 'Student227'), (277, 'Student277'), (327, 'Student327'), (377, 'Student377'), (427, 'Student
427'), (477, 'Student477')]
28 : [(28, 'Student28'), (78, 'Student78'), (128, 'Student128'), (178, 'Student178'), (228, 'Student228'), (278, 'Student278'), (328, 'Student328'), (378, 'Student378'), (428, 'Student
428'), (478, 'Student478')]
29 : [(29, 'Student29'), (79, 'Student79'), (129, 'Student129'), (179, 'Student179'), (229, 'Student229'), (279, 'Student279'), (329, 'Student329'), (379, 'Student379'), (429, 'Student
429'), (479, 'Student479')]
30 : [(30, 'Student30'), (80, 'Student80'), (130, 'Student130'), (180, 'Student180'), (230, 'Student230'), (280, 'Student280'), (330, 'Student330'), (380, 'Student380'), (430, 'Student
430'), (480, 'Student480')]
31 : [(31, 'Student31'), (81, 'Student81'), (131, 'Student131'), (181, 'Student181'), (231, 'Student231'), (281, 'Student281'), (331, 'Student331'), (381, 'Student381'), (431, 'Student
431'), (481, 'Student481')]
32 : [(32, 'Student32'), (82, 'Student82'), (132, 'Student132'), (182, 'Student182'), (232, 'Student232'), (282, 'Student282'), (332, 'Student332'), (382, 'Student382'), (432, 'Student
432'), (482, 'Student482')]
33 : [(33, 'Student33'), (83, 'Student83'), (133, 'Student133'), (183, 'Student183'), (233, 'Student233'), (283, 'Student283'), (333, 'Student333'), (383, 'Student383'), (433, 'Student
433'), (483, 'Student483')]
34 : [(34, 'Student34'), (84, 'Student84'), (134, 'Student134'), (184, 'Student184'), (234, 'Student234'), (284, 'Student284'), (334, 'Student334'), (384, 'Student384'), (434, 'Student
434'), (484, 'Student484')]
35 : [(35, 'Student35'), (85, 'Student85'), (135, 'Student135'), (185, 'Student185'), (235, 'Student235'), (285, 'Student285'), (335, 'Student335'), (385, 'Student385'), (435, 'Student
435'), (485, 'Student485')]
36 : [(36, 'Student36'), (86, 'Student86'), (136, 'Student136'), (186, 'Student186'), (236, 'Student236'), (286, 'Student286'), (336, 'Student336'), (386, 'Student386'), (436, 'Student
436'), (486, 'Student486')]
37 : [(37, 'Student37'), (87, 'Student87'), (137, 'Student137'), (187, 'Student187'), (237, 'Student237'), (287, 'Student287'), (337, 'Student337'), (387, 'Student387'), (437, 'Student
437'), (487, 'Student487')]
38 : [(38, 'Student38'), (88, 'Student88'), (138, 'Student138'), (188, 'Student188'), (238, 'Student238'), (288, 'Student288'), (338, 'Student338'), (388, 'Student388'), (438, 'Student
438'), (488, 'Student488')]
39 : [(39, 'Student39'), (89, 'Student89'), (139, 'Student139'), (189, 'Student189'), (239, 'Student239'), (289, 'Student289'), (339, 'Student339'), (389, 'Student389'), (439, 'Student
439'), (489, 'Student489')]
40 : [(40, 'Student40'), (90, 'Student90'), (140, 'Student140'), (190, 'Student190'), (240, 'Student240'), (290, 'Student290'), (340, 'Student340'), (390, 'Student390'), (440, 'Student
440'), (490, 'Student490')]
41 : [(41, 'Student41'), (91, 'Student91'), (141, 'Student141'), (191, 'Student191'), (241, 'Student241'), (291, 'Student291'), (341, 'Student341'), (391, 'Student391'), (441, 'Student
441'), (491, 'Student491')]
42 : [(42, 'Student42'), (92, 'Student92'), (142, 'Student142'), (192, 'Student192'), (242, 'Student242'), (292, 'Student292'), (342, 'Student342'), (392, 'Student392'), (442, 'Student
442'), (492, 'Student492')]
43 : [(43, 'Student43'), (93, 'Student93'), (143, 'Student143'), (193, 'Student193'), (243, 'Student243'), (293, 'Student293'), (343, 'Student343'), (393, 'Student393'), (443, 'Student
443'), (493, 'Student493')]
44 : [(44, 'Student44'), (94, 'Student94'), (144, 'Student144'), (194, 'Student194'), (244, 'Student244'), (294, 'Student294'), (344, 'Student344'), (394, 'Student394'), (444, 'Student
444'), (494, 'Student494')]
45 : [(45, 'Student45'), (95, 'Student95'), (145, 'Student145'), (195, 'Student195'), (245, 'Student245'), (295, 'Student295'), (345, 'Student345'), (395, 'Student395'), (445, 'Student
445'), (495, 'Student495')]
46 : [(46, 'Student46'), (96, 'Student96'), (146, 'Student146'), (196, 'Student196'), (246, 'Student246'), (296, 'Student296'), (346, 'Student346'), (396, 'Student396'), (446, 'Student
446'), (496, 'Student496')]
47 : [(47, 'Student47'), (97, 'Student97'), (147, 'Student147'), (197, 'Student197'), (247, 'Student247'), (297, 'Student297'), (347, 'Student347'), (397, 'Student397'), (447, 'Student
447'), (497, 'Student497')]
48 : [(48, 'Student48'), (98, 'Student98'), (148, 'Student148'), (198, 'Student198'), (248, 'Student248'), (298, 'Student298'), (348, 'Student348'), (398, 'Student398'), (448, 'Student
448'), (498, 'Student498')]
49 : [(49, 'Student49'), (99, 'Student99'), (149, 'Student149'), (199, 'Student199'), (249, 'Student249'), (299, 'Student299'), (349, 'Student349'), (399, 'Student399'), (449, 'Student
449'), (499, 'Student499')]
>>>
```

Ln: 59 Col: 0

Question 3: Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

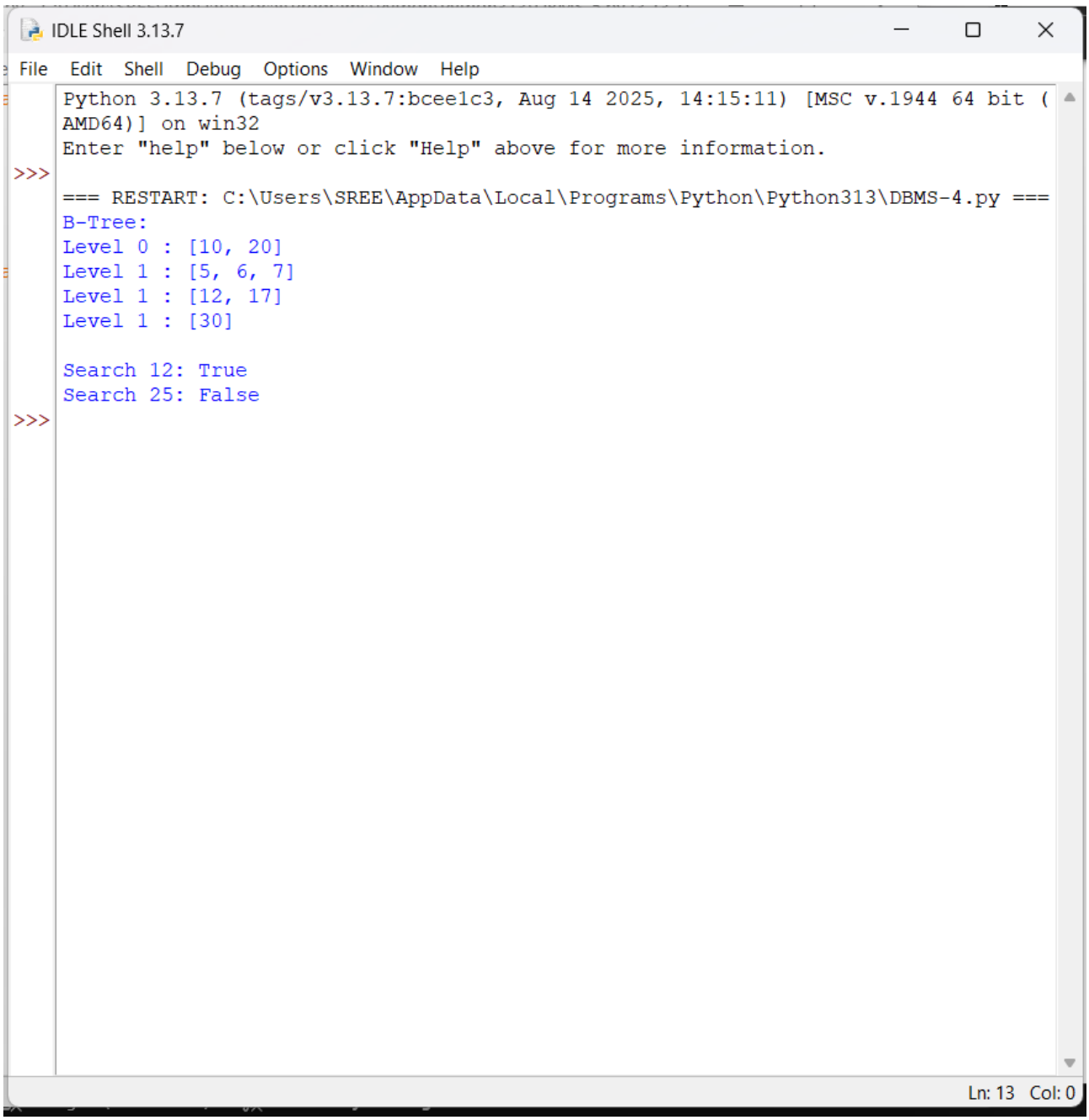
```
class SortedFile:
    def __init__(self):
        self.records = []

    def insert(self, student_id, name):
        self.records.append((student_id, name))
        self.records.sort()

    def binary_search(self, student_id):
        low = 0
        high = len(self.records) - 1

        while low <= high:
            mid = (low + high) // 2

            if self.records[mid][0] == student_id:
                return self.records[mid]

            elif self.records[mid][0] < student_id:
                low = mid + 1

            else:
                high = mid - 1

        return None

    def display(self):
        for record in self.records:
            print(record)


# Main program
file = SortedFile()

file.insert(105, "Ravi")
file.insert(101, "Arun")
file.insert(110, "Sita")
file.insert(103, "Kiran")
file.insert(108, "Priya")

print("Sorted File:")
file.display()

print("\nSearch Result:")
print(file.binary_search(108))
```


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:\Users\SREE\AppData\Local\Programs\Python\Python313\DBMS-1.py ===
Inserted: {'id': 101, 'name': 'Ravi', 'marks': 85}
Inserted: {'id': 102, 'name': 'Sita', 'marks': 90}
Inserted: {'id': 103, 'name': 'Arun', 'marks': 78}
Sequential Scan:
{'id': 101, 'name': 'Ravi', 'marks': 85}
{'id': 102, 'name': 'Sita', 'marks': 90}
{'id': 103, 'name': 'Arun', 'marks': 78}
Deleted record with ID: 102
Sequential Scan:
{'id': 101, 'name': 'Ravi', 'marks': 85}
{'id': 103, 'name': 'Arun', 'marks': 78}
>>>
=== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/DBMS-3.py ===
Sorted File:
(101, 'Arun')
(103, 'Kiran')
(105, 'Ravi')
(108, 'Priya')
(110, 'Sita')
Search Result:
(108, 'Priya')
>>>
Ln: 29 Col: 0

Question 4: Implement a B-Tree of order 4 in code. Support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17..

```
class BTreeNode:
    def __init__(self, leaf=False):
        self.keys = []
        self.children = []
        self.leaf = leaf


class BTree:
    def __init__(self, order=4):
        self.root = BTreeNode(True)
        self.order = order

    def search(self, node, key):
        i = 0

        while i < len(node.keys) and key > node.keys[i]:
            i += 1

        if i < len(node.keys) and key == node.keys[i]:
            return True

        if node.leaf:
            return False

        return self.search(node.children[i], key)

    def split_child(self, parent, index):
        child = parent.children[index]
        new_node = BTreeNode(child.leaf)

        middle = child.keys[1]

        new_node.keys = child.keys[2:]
        child.keys = child.keys[:1]

        if not child.leaf:
            new_node.children = child.children[2:]
            child.children = child.children[:2]

        parent.keys.insert(index, middle)
        parent.children.insert(index + 1, new_node)

    def insert(self, key):
        root = self.root

        if len(root.keys) == self.order - 1:
            new_root = BTreeNode(False)
            new_root.children.append(root)
            self.root = new_root

            self.split_child(new_root, 0)
```

```
            self.insert_non_full(new_root, key)
        else:
            self.insert_non_full(root, key)

    def insert_non_full(self, node, key):
        i = len(node.keys) - 1

        if node.leaf:
            node.keys.append(0)

            while i >= 0 and key < node.keys[i]:
                node.keys[i + 1] = node.keys[i]
                i -= 1

            node.keys[i + 1] = key

        else:
            while i >= 0 and key < node.keys[i]:
                i -= 1

            i += 1

            if len(node.children[i].keys) == self.order - 1:
                self.split_child(node, i)

                if key > node.keys[i]:
                    i += 1

            self.insert_non_full(node.children[i], key)

    def display(self, node=None, level=0):
        if node is None:
            node = self.root

        print("Level", level, ":", node.keys)

        if not node.leaf:
            for child in node.children:
                self.display(child, level + 1)


# Main program
tree = BTree(4)

keys = [10, 20, 5, 6, 12, 30, 7, 17]

for key in keys:
    tree.insert(key)

print("B-Tree:")
tree.display()

print("\nSearch 12:", tree.search(tree.root, 12))
print("Search 25:", tree.search(tree.root, 25))
```


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:\Users\SREE\AppData\Local\Programs\Python\Python313\DBMS-4.py ===
B-Tree:
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]
Search 12: True
Search 25: False
>>>
Ln: 13 Col: 0

Question 5: Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

Answer:

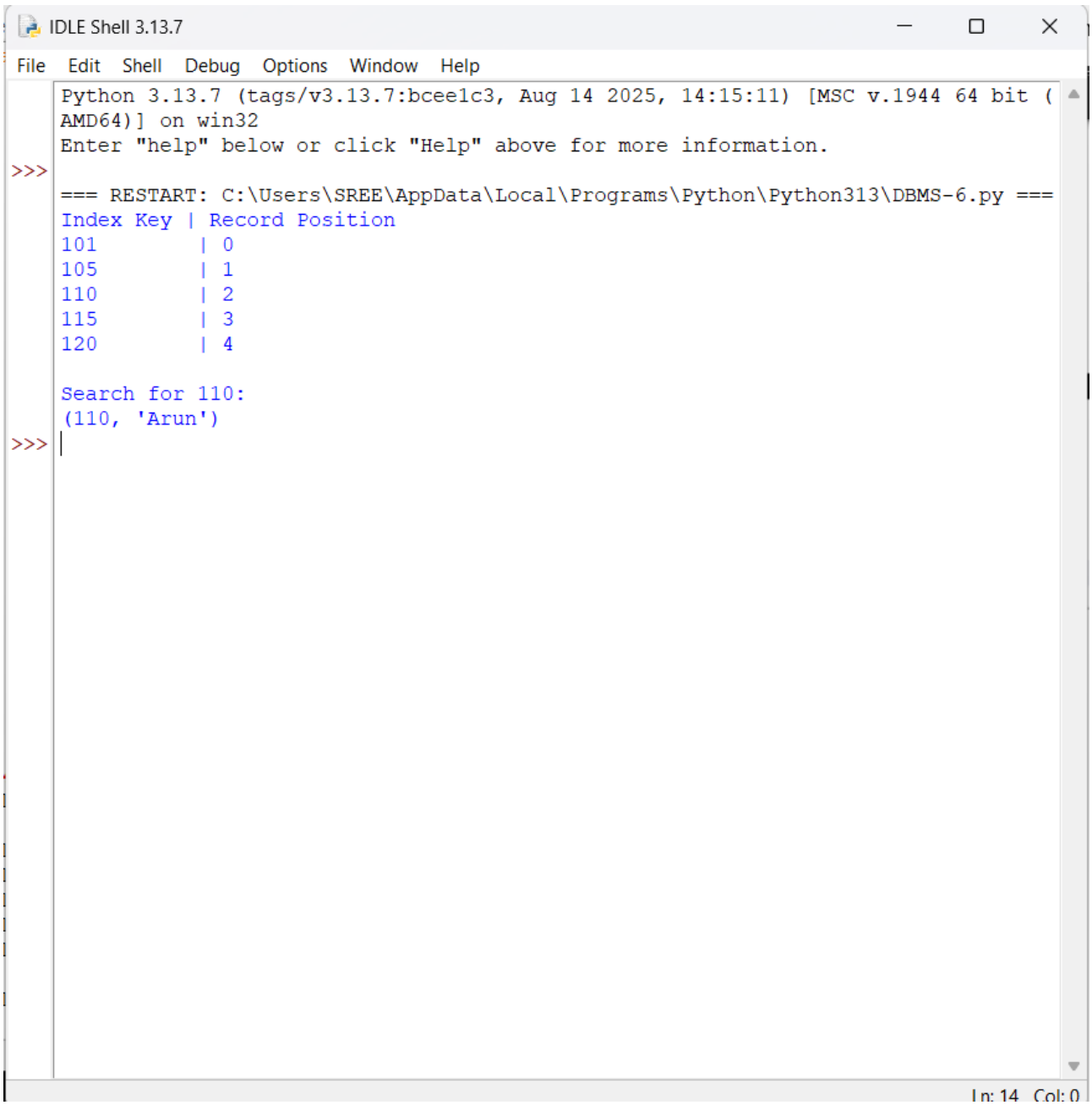
```
class BPlusNode:
        else:
            parent_key = child.keys[mid]
            new_node.keys = child.keys[mid + 1:]
            child.keys = child.keys[:mid]

            new_node.children = child.children[mid + 1:]
            child.children = child.children[:mid + 1]

            parent.keys.insert(index, parent_key)

        parent.children.insert(index + 1, new_node)

    def insert_non_full(self, node, key):
        if node.leaf:
            node.keys.append(key)
            node.keys.sort()
        else:
            i = len(node.keys) - 1

            while i >= 0 and key < node.keys[i]:
                i -= 1

            i += 1

            if len(node.children[i].keys) == self.order:
                self.split_child(node, i)

                if key >= node.keys[i]:
                    i += 1

            self.insert_non_full(node.children[i], key)

    def find_leaf(self, key):
        node = self.root

        while not node.leaf:
            i = 0

            while i < len(node.keys) and key >= node.keys[i]:
                i += 1

            node = node.children[i]

        return node

    def range_query(self, start, end):
        node = self.find_leaf(start)
        result = []

        while node:
```

```
            for key in node.keys:
                if start <= key <= end:
                    result.append(key)

                if key > end:
                    return result

            node = node.next

        return result

    def display_leaves(self):
        node = self.root

        while not node.leaf:
            node = node.children[0]

        print("Leaf Level:")

        while node:
            print(node.keys, end=" -> ")
            node = node.next

        print("None")


# Main program
tree = BPlusTree()

for key in [10, 20, 5, 15, 25, 30, 35, 40, 45, 50]:
    tree.insert(key)

tree.display_leaves()

print("Range 15 to 40:")
print(tree.range_query(15, 40))
```


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:\Users\SREE\AppData\Local\Programs\Python\Python313\DBMS-5.py ===
Leaf Level:
[5] -> [10] -> [15] -> [20] -> [25] -> [30] -> [35] -> [40, 45, 50] -> None
Range 15 to 40:
[15, 20, 25, 30, 35, 40]
>>>
Ln: 9 Col: 0

Question 6: Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

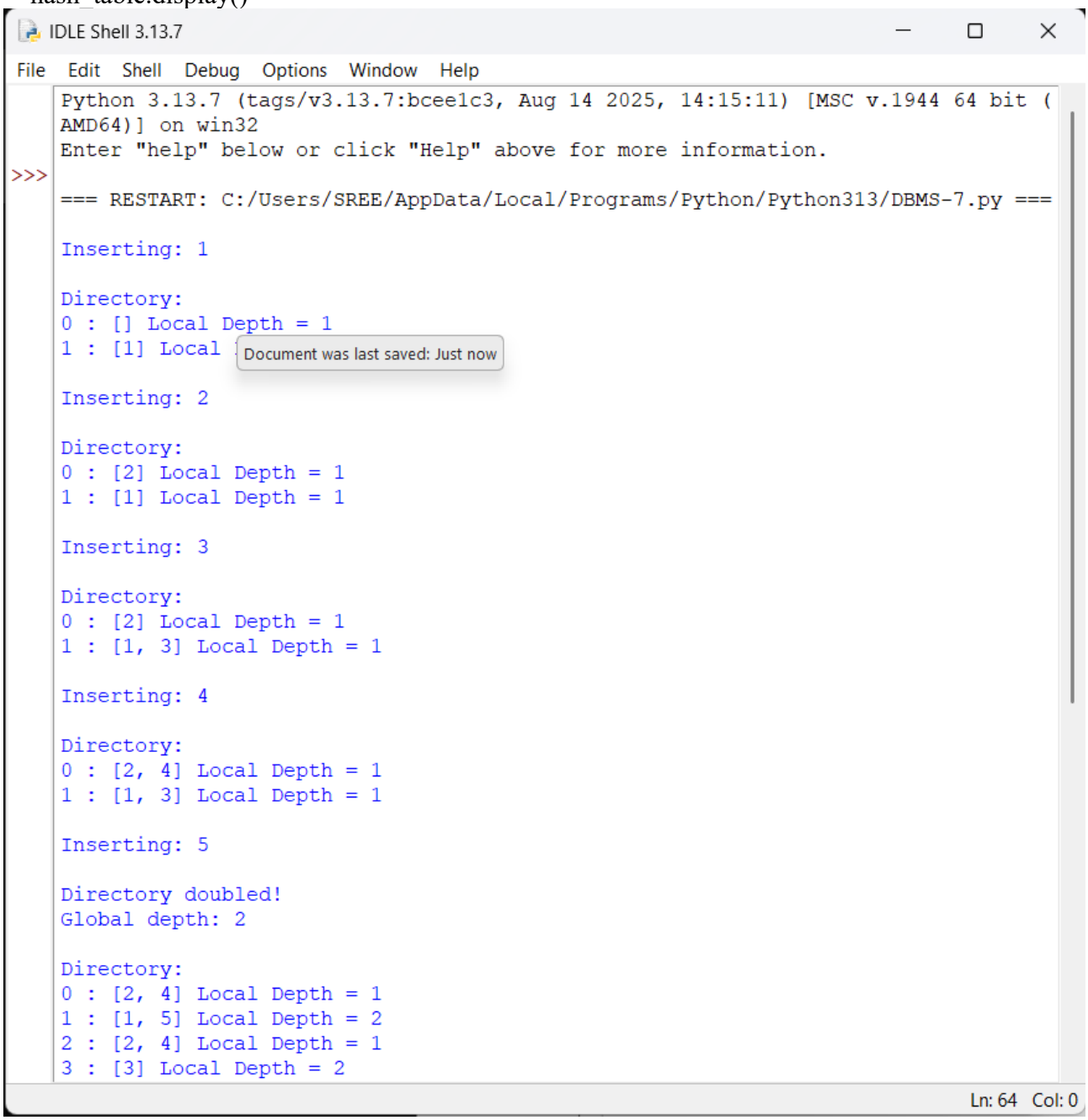
```
class DenseIndex:
    def __init__(self):
        self.records = []
        self.index = []

    def insert(self, key, value):
        self.records.append((key, value))
        self.records.sort()

    def build_index(self):
        self.index = []

        for position, record in enumerate(self.records):
            self.index.append((record[0], position))

    def search(self, key):
        for index_key, position in self.index:
            if index_key == key:
                return self.records[position]

        return None

    def display_index(self):
        print("Index Key | Record Position")

        for key, position in self.index:
            print(key, "       |", position)


# Main program
file = DenseIndex()

file.insert(101, "Ravi")
file.insert(105, "Sita")
file.insert(110, "Arun")
file.insert(115, "Kiran")
file.insert(120, "Priya")

file.build_index()

file.display_index()

print("\nSearch for 110:")
print(file.search(110))
```


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:\Users\SREE\AppData\Local\Programs\Python\Python313\DBMS-6.py ===
Index Key | Record Position
101 | 0
105 | 1
110 | 2
115 | 3
120 | 4
Search for 110:
(110, 'Arun')
>>>
Ln: 14 Col: 0

Question 7: Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

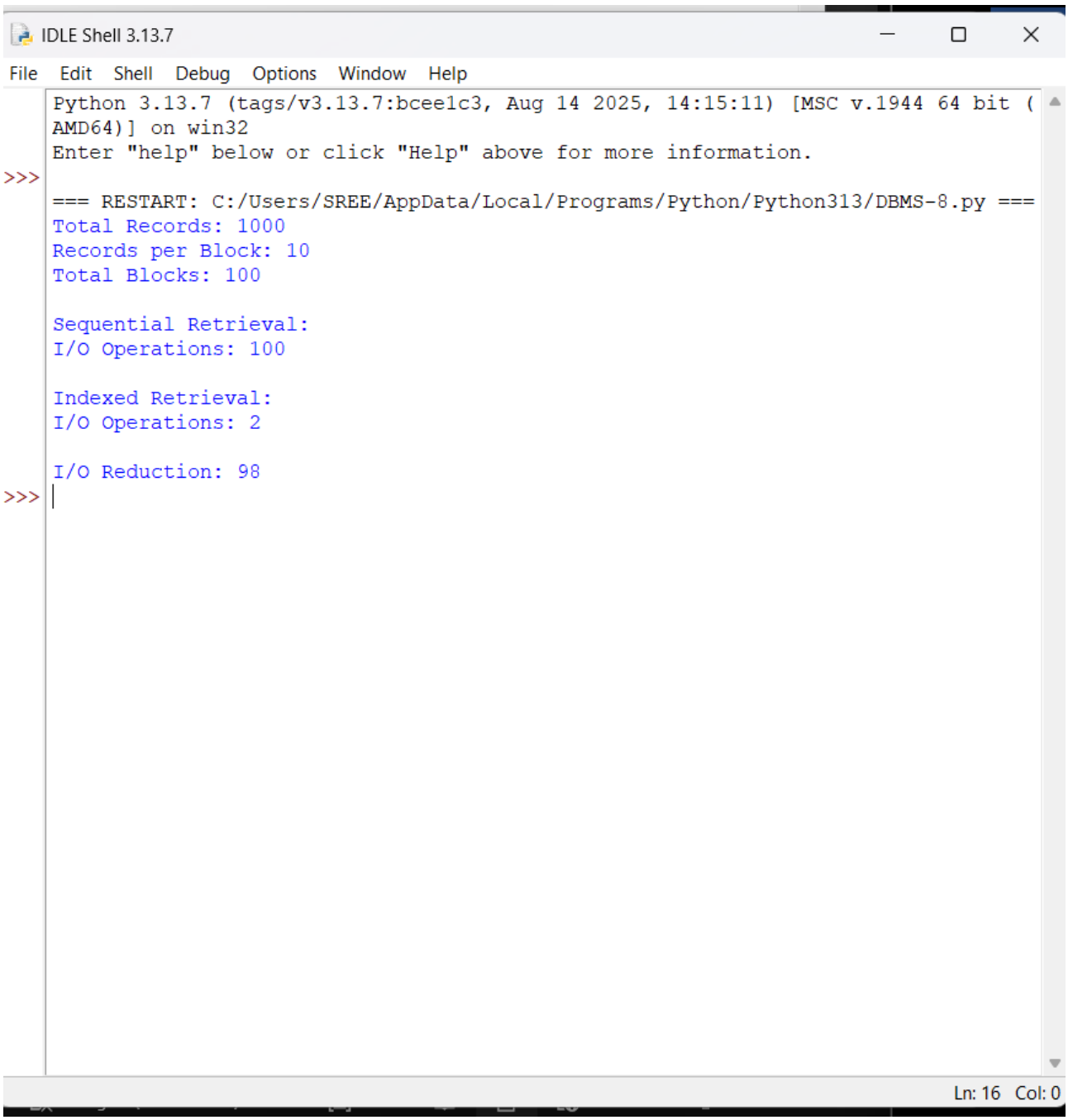
```
class Bucket:
    def __init__(self, depth):
        self.depth = depth
        self.records = []


class ExtendibleHash:
    def __init__(self, bucket_size=2):
        self.bucket_size = bucket_size
        self.global_depth = 1

        self.directory = [
            Bucket(1),
            Bucket(1)
        ]

    def hash_function(self, key):
        return key

    def insert(self, key):
        index = key & ((1 << self.global_depth) - 1)
        bucket = self.directory[index]

        if len(bucket.records) < self.bucket_size:
            bucket.records.append(key)
            return

        if bucket.depth == self.global_depth:
            old_directory = self.directory

            self.global_depth += 1

            self.directory = old_directory + old_directory.copy()

            print("\nDirectory doubled!")
            print("Global depth:", self.global_depth)

        bucket.depth += 1

        old_records = bucket.records
        bucket.records = []

        new_bucket = Bucket(bucket.depth)

        index = self.directory.index(bucket)

        split_index = index ^ (1 << (bucket.depth - 1))
        self.directory[split_index] = new_bucket

        for record in old_records:
            new_index = record & ((1 << self.global_depth) - 1)
```

```
            self.directory[new_index].records.append(record)

        self.insert(key)

    def display(self):
        print("\nDirectory:")
        for i, bucket in enumerate(self.directory):
            print(i, ":", bucket.records,
                  "Local Depth =", bucket.depth)


# Main program
hash_table = ExtendibleHash(2)

for key in [1, 2, 3, 4, 5, 6, 7, 8]:
    print("\nInserting:", key)
    hash_table.insert(key)
    hash_table.display()
```



Question 8: Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

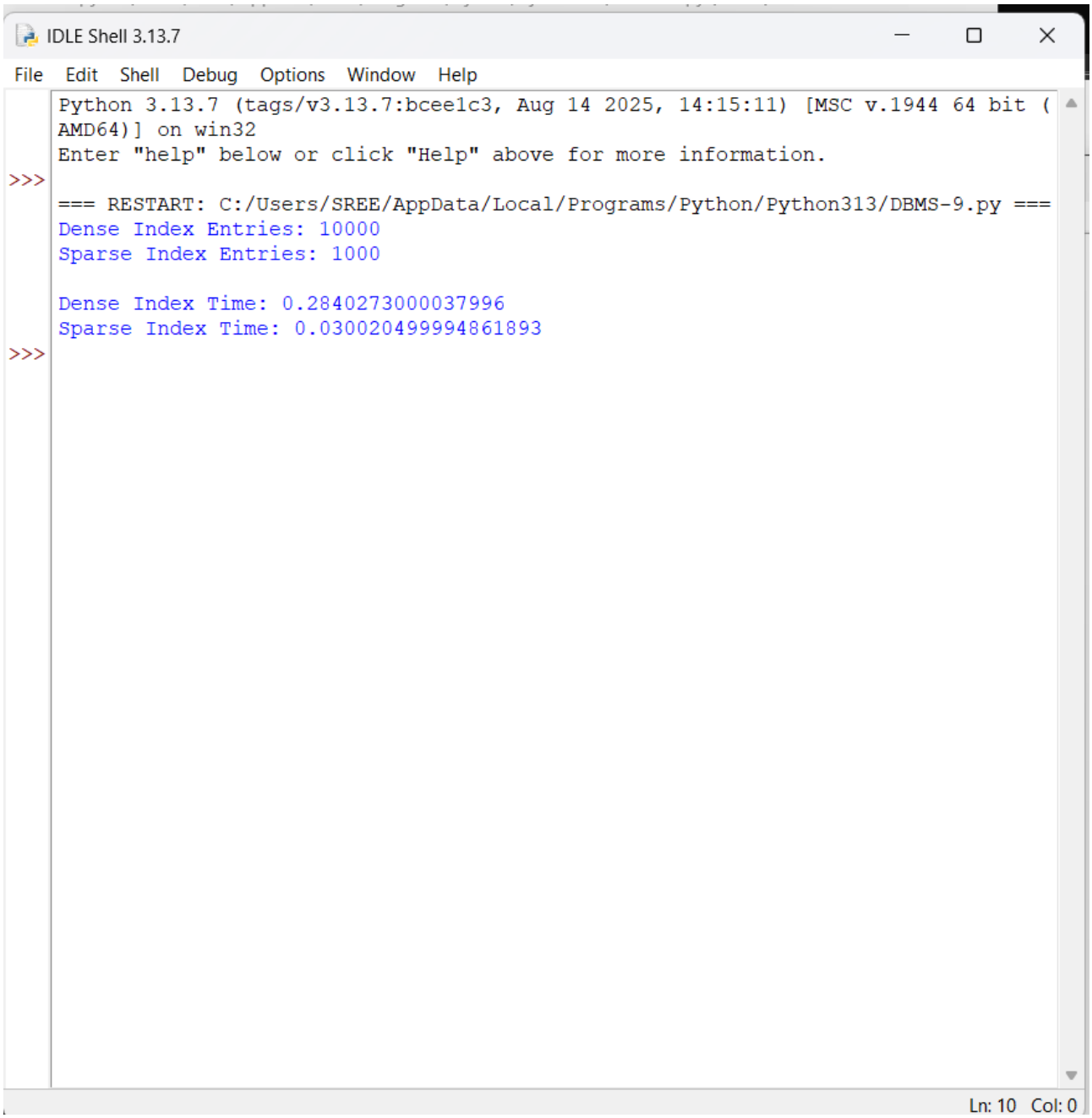
```
import math

# Total records and records per block
records = 1000
records_per_block = 10

total_blocks = math.ceil(records / records_per_block)

# Sequential retrieval
sequential_io = total_blocks

# Indexed retrieval
# Assume one index block followed by one data block
indexed_io = 2

print("Total Records:", records)
print("Records per Block:", records_per_block)
print("Total Blocks:", total_blocks)

print("\nSequential Retrieval:")
print("I/O Operations:", sequential_io)

print("\nIndexed Retrieval:")
print("I/O Operations:", indexed_io)

print("\nI/O Reduction:",
      sequential_io - indexed_io)
```


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/DBMS-8.py ===
Total Records: 1000
Records per Block: 10
Total Blocks: 100
Sequential Retrieval:
I/O Operations: 100
Indexed Retrieval:
I/O Operations: 2
I/O Reduction: 98
>>>
Ln: 16 Col: 0

Question 9: Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

```
import time
import random
import bisect

# Create sorted records
records = [(i, "Student" + str(i)) for i in range(1, 10001)]

# Dense index
dense_index = [(key, pos) for pos, (key, name) in enumerate(records)]

# Sparse index: one index entry for every 10 records
sparse_index = [
    (records[i][0], i)
    for i in range(0, len(records), 10)
]


def dense_search(key):
    keys = [x[0] for x in dense_index]
    pos = bisect.bisect_left(keys, key)

    if pos < len(keys) and keys[pos] == key:
        return records[dense_index[pos][1]]

    return None


def sparse_search(key):
    keys = [x[0] for x in sparse_index]
    pos = bisect.bisect_right(keys, key) - 1

    if pos < 0:
        pos = 0

    start = sparse_index[pos][1]
    end = min(start + 10, len(records))

    for i in range(start, end):
        if records[i][0] == key:
            return records[i]

    return None


keys = [random.randint(1, 10000) for _ in range(1000)]

start = time.perf_counter()

for key in keys:
    dense_search(key)

dense_time = time.perf_counter() - start
```

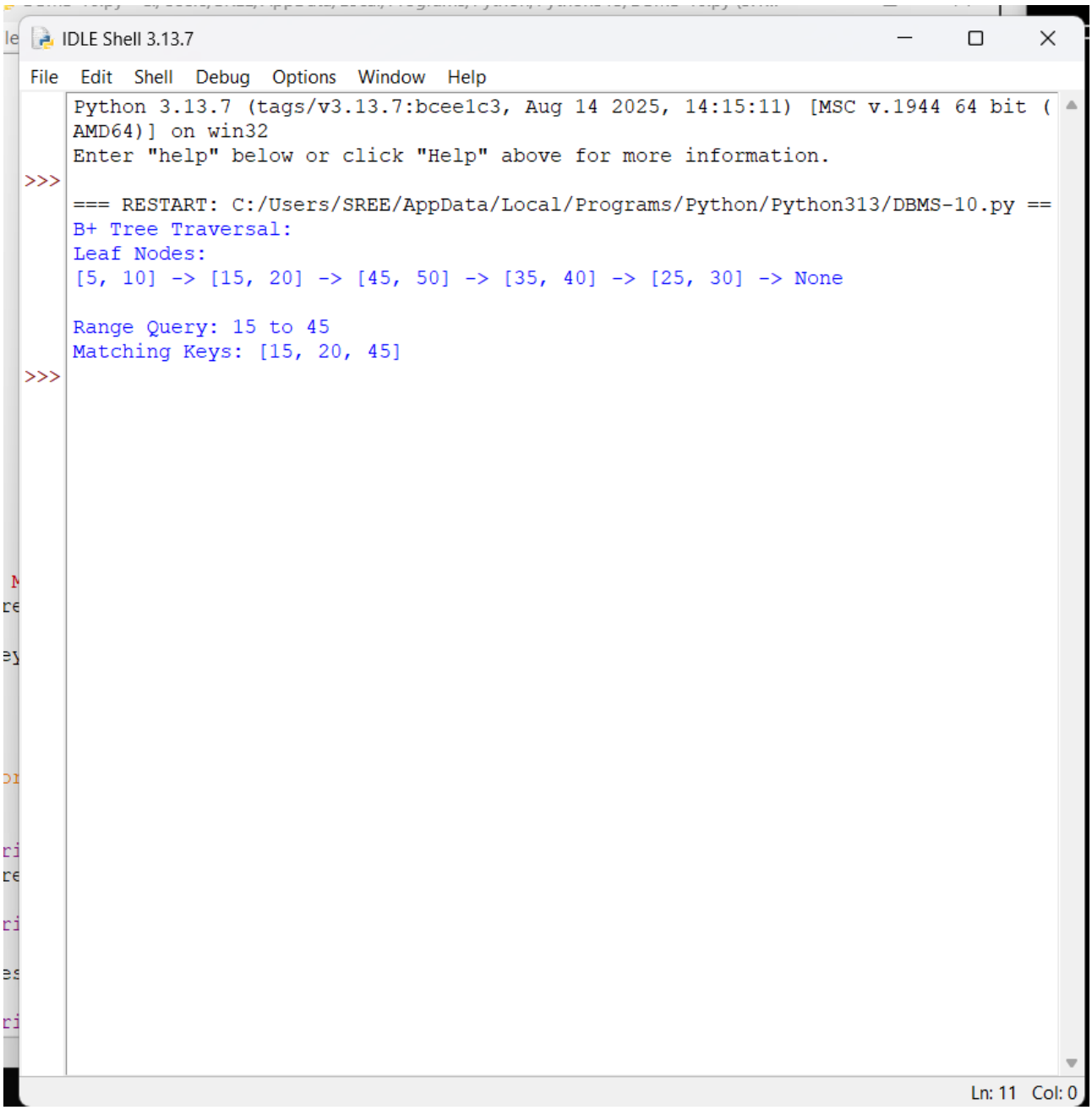
```
start = time.perf_counter()

for key in keys:
    sparse_search(key)

sparse_time = time.perf_counter() - start


print("Dense Index Entries:", len(dense_index))
print("Sparse Index Entries:", len(sparse_index))

print("\nDense Index Time:",
      dense_time)

print("Sparse Index Time:",
      sparse_time)
```



Question 10: Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching keys.

```
class Node:
    def __init__(self, leaf=False):
        self.keys = []
        self.children = []
        self.leaf = leaf
        self.next = None


class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.root = Node(True)

    def insert(self, key):
        leaf = self.find_leaf(key)

        leaf.keys.append(key)
        leaf.keys.sort()

        if len(leaf.keys) >= self.order:
            self.split_leaf(leaf)

    def find_leaf(self, key):
        node = self.root

        while not node.leaf:
            i = 0

            while i < len(node.keys) and key >= node.keys[i]:
                i += 1

            node = node.children[i]

        return node

    def split_leaf(self, leaf):
        new_leaf = Node(True)

        mid = len(leaf.keys) // 2

        new_leaf.keys = leaf.keys[mid:]
        leaf.keys = leaf.keys[:mid]

        new_leaf.next = leaf.next
        leaf.next = new_leaf

        if leaf == self.root:
            new_root = Node(False)

            new_root.keys = [new_leaf.keys[0]]
            new_root.children = [leaf, new_leaf]
```

```
            self.root = new_root

    def traverse(self):
        node = self.root

        while not node.leaf:
            node = node.children[0]

        print("Leaf Nodes:")

        while node:
            print(node.keys, end=" -> ")
            node = node.next

        print("None")

    def range_query(self, start, end):
        node = self.find_leaf(start)
        result = []

        while node:
            for key in node.keys:
                if start <= key <= end:
                    result.append(key)

                if key > end:
                    return result

            node = node.next

        return result


# Main program
tree = BPlusTree()

keys = [
    5, 10, 15, 20, 25,
    30, 35, 40, 45, 50
]

for key in keys:
    tree.insert(key)

print("B+ Tree Traversal:")
tree.traverse()

print("\nRange Query: 15 to 45")

result = tree.range_query(15, 45)

print("Matching Keys:", result)
```


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/DBMS-10.py ==
B+ Tree Traversal:
Leaf Nodes:
[5, 10] -> [15, 20] -> [45, 50] -> [35, 40] -> [25, 30] -> None
Range Query: 15 to 45
Matching Keys: [15, 20, 45]
>>>
Ln: 11 Col: 0